Filtering with django-filter

This filter concept is a cool idea, and it can be applied to your regular views as well. You could go through the effort of handling all the query arguments to provide filters on your views, or you could use django-filter (https://pypi.org/project/django-filter/). This third-party library works similarly to Ninja's `FilterSchema`: you build a `django_filters.FilterSet` class and then use it to fetch operate on your `QuerySet` in a view, passing in the request and all the possible query parameters. With just a few lines of code, any of your Django views can support filtering.

11.2.4 Nesting data

Sometimes, even your data has data. Consider the rooms that belong to venues. You could write an API endpoint that lists rooms that took a venue ID as an argument, but then, determining the rooms that belong to a venue would take two calls. A list of all venues and their corresponding rooms would take $N + 1$ calls, one to list the venues and N more for each of the corresponding rooms. This is expensive on the network and, for a long listing, hard on the database in the backend. A better solution is to nest the room information inside the venue data.

Ninja makes data nesting straightforward: add a serializer for the nested object, and set an appropriate Python hint in the enclosing one. Recall that each `Room` model gets connected to a `Venue` through a many-to-many relationship. Each `Venue` object has a `.room_set` query manager that you can use to find associated rooms. Ninja knows how to do this. By defining a schema for a room and adding a `room_set` field to `VenueOut`, Ninja automatically performs the subquery and nests the data.

The name `room_set` is very Django-esque. Developers using your API might not understand what that means, and *rooms* would be easier to understand. Ninja allows you to rename fields using the `Field` class—in fact, that's what was going on when you aliased the `name__istartswith` query parameter. The following listing contains the code for a room serializer and the change to `VenueOut` to include the nested data.

Listing 11.11 Nesting `Room` data inside serialized `Venue` objects

```
# RiffMates/bands/api.py
...
from bands.models import Venue, Room

...

class RoomSchema(ModelSchema):
    class Meta:
        model = Room
        fields = ["id", "name"]

class VenueOut(ModelSchema):
    slug: str
    url: str
```

Add the Room model to your imports.

Create a serialization schema for rooms.